

TP01 – Pontes e Caminho Euleriano

Métodos Naïve e Tarjan + Algoritmo de Fleury

PUC Minas – Ciência da Computação
Teoria dos Grafos e Computabilidade
Prof. Zenilton Kleber Gonçalves do Patrocínio Júnior

April 26, 2026

1 Introdução

Uma ponte é uma aresta cuja remoção desconecta o grafo. O conceito aparece em vários problemas, e o que motivou esse trabalho foi o algoritmo de Fleury, que constrói caminhos eulerianos – sequências que percorrem cada aresta do grafo exatamente uma vez. A cada passo o Fleury precisa saber se uma aresta candidata é ponte; portanto, a estratégia escolhida para detectar pontes muda o comportamento prático do algoritmo.

Aqui implementamos os dois métodos pedidos pelo enunciado, o naïve (que testa conectividade após remover cada aresta) e o de Tarjan (1974), que faz tudo em uma única DFS calculando *lowlink*. Os dois são acoplados ao Fleury via uma interface, o que permite trocar a estratégia sem mexer no resto. Em seguida, comparamos tempo médio em grafos aleatórios de 100, 1.000, 10.000 e 100.000 vértices, nos três tipos eulerianos.

2 Fundamentos

2.1 Quando existe caminho ou circuito euleriano

Pelo teorema de Euler, num grafo conexo:

- existe circuito euleriano sse todos os vértices têm grau par;
- existe caminho euleriano (sem ser circuito) sse exatamente dois vértices têm grau ímpar;
- caso contrário, nem caminho nem circuito existem.

Essa classificação é a primeira coisa que o Fleury faz. Se o grafo cai no terceiro caso, abortamos antes de tentar construir o caminho.

2.2 Método naïve

O naïve é literalmente a definição de ponte virada em código. Para cada aresta (u, v) , removemos, perguntamos se v ainda é alcançável a partir de u por DFS, e reinserimos. Se a resposta foi não, (u, v) era ponte. O custo é $O(E \cdot (V + E))$, uma DFS por aresta.

Uma otimização que usamos: em vez de `isConnected()` (que varre o grafo todo), o naïve usa `isReachable(u, v)`, que para a DFS assim que encontra v . Em grafos esparsos, essa terminação antecipada faz uma diferença grande, e isso vai reaparecer na análise dos experimentos.

2.3 Método de Tarjan

A ideia do Tarjan é trocar E DFS por uma só. Em uma única passada, mantemos para cada vértice v :

- $\text{disc}(v)$, o instante em que v foi descoberto;
- $\text{low}(v)$, o menor disc alcançável a partir de v usando arestas da árvore DFS e no máximo uma aresta de retorno.

Quando v é filho de u na árvore DFS, a aresta (u, v) é ponte se e somente se $\text{low}(v) > \text{disc}(u)$. Em palavras: nada que sai da subárvore de v chega em u ou em algum ancestral de u por outro caminho. Custo $O(V + E)$.

Implementamos a versão iterativa, com pilha explícita. A versão recursiva existe no código (`TarjanBridgeFinderRec.java`) só por valor didático; em $V \geq 10.000$ ela estoura o stack do Java.

2.4 Fleury

O algoritmo, na prática, é:

1. classifique o grafo; se for não-euleriano, aborte;
2. escolha v_0 : ímpar (semi-euleriano) ou qualquer vértice de grau positivo (euleriano);
3. estando em u , prefira sempre uma aresta que não seja ponte. Pontes são último recurso;
4. remova a aresta percorrida, mova-se para o vizinho, repita até esgotar arestas.

A regra de evitar pontes é o que evita ficar preso. Atravessar uma ponte sem necessidade pode isolar arestas que ainda precisariam ser percorridas, e aí o algoritmo trava antes do fim.

3 Implementação

Tudo em Java 21. Os arquivos estão organizados assim:

Table 1: Arquivos do projeto

Arquivo	O que faz
<code>Graph.java</code>	Lista de adjacência, contador de versão e de arestas
<code>GraphUtils.java</code>	DFS, BFS, conectividade, classificação euleriana
<code>GraphGenerator.java</code>	Gera grafos aleatórios dos três tipos
<code>BridgeFinder.java</code>	Interface (<i>Strategy</i>) para detectar pontes
<code>NaiveBridgeFinder.java</code>	Naïve: remove + testa + reinsere
<code>TarjanBridgeFinder.java</code>	Tarjan iterativo com cache versionado
<code>TarjanBridgeFinderRec.java</code>	Tarjan recursivo (didático, não usado)
<code>FleuryEulerPath.java</code>	Fleury parametrizado por <code>BridgeFinder</code>
<code>Experiments.java</code>	Bateria de experimentos $\rightarrow$ CSV + tabelas
<code>Main.java</code>	Menu interativo + CLI batch

3.1 Estrutura de dados

Lista de adjacência. A alternativa, matriz, ocuparia $O(V^2)$, o que para $V = 100.000$ daria 10^{10} posições – impossível na memória disponível. Lista de adjacência custa $O(V + E)$ e ainda permite remoção/inserção de arestas em tempo razoável, que é o que o naïve precisa fazer o tempo todo.

3.2 Interface comum para pontes

A interface `BridgeFinder` é o ponto de troca de estratégia. O Fleury não sabe se está usando naïve ou Tarjan, só pergunta se a aresta é ponte:

```
public interface BridgeFinder {
    List<int[]> findBridges(Graph g);
    boolean isBridge(Graph g, int u, int v);
    String getName();
}
```

3.3 Cache versionado no Tarjan

O `isBridge` do Tarjan internamente chama `findBridges` (custa $O(V + E)$) e procura a aresta no resultado. O Fleury chama `isBridge` várias vezes seguidas no mesmo estado do grafo, então faz sentido guardar o resultado.

A solução: um contador de versão no `Graph`, incrementado a cada `addEdge/removeEdge`. O Tarjan armazena (versão observada, conjunto de pontes) e só recomputa se a versão atual diferir. Sem isso, Fleury+Tarjan seria $O(\deg \cdot E \cdot (V + E))$; com cache, $O(E \cdot (V + E))$.

4 Experimentos

4.1 Configuração

Linux x86_64, JDK 21, JVM com `-Xss64m -Xmx2g`. Cada configuração roda 3 vezes; reportamos a média. Tamanhos {100, 1.000, 10.000, 100.000} e densidade alvo de aproximadamente 1,1 V arestas (grafo esparso, próximo de uma árvore com algumas arestas extras).

4.2 Tempo médio para encontrar todas as pontes

Table 2: Tempo médio para encontrar todas as pontes do grafo (`findBridges`). “—” significa que a configuração foi pulada por inviabilidade de tempo.

N	Tipo	Arestas	Naïve (ms)	Tarjan (ms)
100	Eulerian	118	4,59	0,24
100	Semi	121	1,50	0,08
100	Não-Euler.	118	1,45	0,08
1.000	Eulerian	1.088	82,40	0,76
1.000	Semi	1.093	43,32	0,65
1.000	Não-Euler.	1.092	44,55	0,63
10.000	Eulerian	10.909	—	1,22
10.000	Semi	10.884	—	1,16
10.000	Não-Euler.	10.916	—	1,24
100.000	Eulerian	108.949	—	19,87
100.000	Semi	109.011	—	19,07
100.000	Não-Euler.	108.982	—	21,17

O resultado é o esperado pela teoria: Tarjan ganha em todos os casos. Em $N = 100$ a diferença é de cerca de 19×; em $N = 1.000$ chega a 108× no caso eulerian (82,4 ms contra 0,76 ms). Em $N = 10.000$ e 100.000 o naïve foi pulado porque a estimativa passa de cinco minutos por execução. Vale notar que o Tarjan resolve o caso de cem mil vértices em cerca de 20 ms.

4.3 Tempo médio do Fleury completo

Table 3: Tempo médio para encontrar o caminho euleriano via Fleury. Em grafos não-eulerianos, o algoritmo aborta na classificação inicial.

N	Tipo	Arestas	Fleury+Naïve (ms)	Fleury+Tarjan (ms)
100	Eulerian	118	2,28	0,91
100	Semi	121	0,53	0,66
100	Não-Euler.	118	0,08	0,07
1.000	Eulerian	1.088	5,23	28,39
1.000	Semi	1.093	5,61	29,62
1.000	Não-Euler.	1.092	0,58	0,55
10.000	Eulerian	10.909	–	20.920,13
10.000	Semi	10.884	–	20.526,37
10.000	Não-Euler.	10.916	–	1,65
100.000	Eulerian	108.949	–	–*
100.000	Semi	109.011	–	–*
100.000	Não-Euler.	108.982	–	–*

* Em $N = 100.000$ a estimativa passa de 30 min por execução: $E \cdot (V + E) \approx 2,2 \times 10^{10}$ operações elementares, o que dá cerca de 18 min com ~ 50 ns por operação, e a tabela exigiria três repetições por tipo.

A leitura da tabela tem alguns pontos interessantes:

Para grafos não-eulerianos, os dois métodos terminam em menos de 1 ms. Faz sentido: a classificação detecta na hora que não existe caminho e o Fleury aborta sem percorrer nada.

Em $N = 100$, Fleury+Tarjan é mais rápido. Em $N = 1.000$, surpresa, Fleury+Naïve foi mais rápido (cerca de 6 ms contra 28 ms). À primeira vista parece contradizer a Tabela 2, mas é coerente quando se olha o uso real:

- o naïve usa `isReachable(u, v)`, que para a DFS assim que encontra v . Em grafos esparsos isso varre poucos vértices;
- o Tarjan, mesmo só pra responder se uma aresta é ponte, precisa terminar a DFS inteira porque computa todas as pontes;
- nosso cache ajuda quando o Fleury olha vários vizinhos do mesmo vértice no mesmo passo, mas a cada `removeEdge` a versão muda e o cache é invalidado.

Em $N = 10.000$ o Fleury+Tarjan gasta cerca de 21 segundos, compatível com a estimativa $E \cdot (V + E) \approx 2 \times 10^8$. Em $N = 100.000$, qualquer Fleury fica inviável, mas a detecção pura de pontes (Tabela 2) continua rodando em 20 ms.

4.4 Por que Tarjan ganha em findBridges mas perde no Fleury?

A pergunta é diferente em cada caso. Quando se quer enumerar todas as pontes do grafo, o Tarjan resolve em uma DFS e o naïve precisa de E delas. Quando se quer só saber se uma aresta específica é ponte, o naïve termina assim que confirma a alcançabilidade, enquanto o Tarjan tem que terminar a DFS para garantir que está computando `low` corretamente.

Ou seja, o método certo depende do que você está perguntando. Para listar pontes, Tarjan domina. Para checar uma de cada vez em grafo esparsos, o naïve compete bem – o que foi um resultado contraintuitivo até a gente parar para pensar no porquê.

5 Como reproduzir

```
# Compilar
javac *.java

# Modo interativo (menu)
java Main

# Experimentos completos (cuidado: pode demorar em N grande)
java Main --experiments

# Experimentos modo seguro (pula naive em N grande)
java Main --experiments-safe
```

Os tempos detalhados são salvos em `resultados/experimentos.csv`.

6 Conclusão

O Tarjan ganha do naïve no que importa para enumerar pontes – até $108\times$ em $N = 1.000$, e é o único viável em $N = 100.000$ (cerca de 20 ms).

No Fleury a comparação é mais sutil. Em grafos esparsos, a terminação antecipada do `isReachable` no naïve compensa a vantagem assintótica do Tarjan, e o naïve às vezes vence. Em qualquer dos dois, o custo total do Fleury é $O(E \cdot (V + E))$, e isso torna o caso $N = 100.000$ inviável – algoritmos modernos como o de Hierholzer ($O(V + E)$) seriam preferíveis para esse tamanho. Mas o enunciado pediu Fleury, e Fleury foi o que fizemos.

References

- [1] TARJAN, R. E. *A note on finding the bridges of a graph*. Information Processing Letters, v. 2, n. 6, p. 160–161, 1974. [https://doi.org/10.1016/0020-0190\(74\)90003-9](https://doi.org/10.1016/0020-0190(74)90003-9).
- [2] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. MIT Press, 2009.
- [3] EULER, L. *Solutio problematis ad geometriam situs pertinentis*. Comment. Acad. Sci. U. Petrop., 1741.